

Algorithms & Data Structures

Week 03

Ahmet Feyzi Ateş

General Information (1st Phase)

- Data, memory, and CPU relationship
- Algorithmic thinking and problem-solving approach
- How search and sort algorithms work
- Time and memory complexity of algorithms
- Solution analysis with **Big-O** notation
- Evaluating and improving code performance
- Stack, Queue, and Hash Table logic
- Recursion and recursive thinking
- Introduction to Dynamic Programming
- *Practical examples from different programming languages*

Finding Maximum value in a given sequence of values

```
Algorithm arrayMax ( $A[0..n-1]$ )  
  currentMax  $\leftarrow A[0]$   
  for  $i \leftarrow 1$  to  $n - 1$  do  
    if  $A[i] > \text{currentMax}$  then  
      currentMax  $\leftarrow A[i]$   
  return currentMax
```

We want to estimate time and space complexity of the above algorithm.

Counting Primitive Operations

- By inspecting the pseudocode, we can determine the maximum number of primitive operations executed by an algorithm, as a function of the input size

Algorithm <i>arrayMax</i> ($A[0..n-1]$)	# operations
$currentMax \leftarrow A[0]$	2 (1 array index, 1 assignment)
for $i \leftarrow 1$ to $n - 1$ do	$n + 1$ (1 assignment, $n-1$ comparisons, 1 jump)
if $A[i] > currentMax$ then	$2(n - 1)$ (1 array index, 1 conditional)
$currentMax \leftarrow A[i]$	$2(n - 1)$ (1 array index, 1 assignment)
<i>{increment counter i ($i \leftarrow i+1$)}</i>	$2(n - 1)$ (1 addition, 1 assignment)
return $currentMax$	1 (1 return)

Total $7n - 2$

Note that the for loop above is iterated $n-1$ times, as i takes values from 1 to $n-1$, inclusive.

*The **for** statement itself is visited $n-1$ times, with a single assignment ($i \leftarrow 1$) during first visit, and one comparison ($i \leq n-1$) during each visit, plus one jump out of the loop at the end. Hence;*

$$1 + n - 1 + 1 = n + 1 \text{ (the increment of loop counter given separately)}$$

Estimating Running Time

- Algorithm *arrayMax* executes $7n - 2$ primitive operations in the worst case. Define:
 - a = Time taken by the fastest primitive operation
 - b = Time taken by the slowest primitive operation
- Let $T(n)$ be worst-case time of *arrayMax*. Then
$$a(7n - 2) \leq T(n) \leq b(7n - 2)$$
- Hence, the running time $T(n)$ is bounded by two linear functions

Types of formulas for basic operation's count

- Exact formula, such as $7n - 2$, i.e.;

$$\begin{aligned} \text{e.g., } T(n) &= n(n-1)/2 \quad \text{or} \quad (n-1)/2, \text{ etc.} \\ &= \tfrac{1}{2}n^2 - \tfrac{1}{2}n \quad \text{or} \quad \tfrac{1}{2}n - \tfrac{1}{2} \end{aligned}$$

- Formula indicating order of growth with specific multiplicative constant:

$$\begin{aligned} \text{e.g., } T(n) &= \tfrac{1}{2}n^2 - \tfrac{1}{2}n \approx \tfrac{1}{2}n^2 = \mathbf{0.5 \, n^2} \\ T(n) &= \tfrac{1}{2}n - \tfrac{1}{2} \approx \tfrac{1}{2}n = \mathbf{0.5 \, n} \end{aligned}$$

- Formula indicating order of growth with unknown multiplicative constant:

$$\text{e.g., } T(n) \approx cn^2$$

Order of growth of Running Time

- **Remember:** we are interested in question like:
 - *How much faster would this algorithm run on a machine that is 10 times faster than the one we have?*
 - *how much longer will the algorithm run if we double its input size?*
- **Most important:** Order of growth within a constant multiple as $n \rightarrow \infty$
- Changing the hardware/ software environment:
 - Affects $T(n)$ by a constant factor, but
 - Does **not** alter the growth rate of $T(n)$
- The **growth rate** of the running time $T(n)$ is an intrinsic property of the algorithm itself.

“Order of Growth” Classification

- Two functions f and g have the relation $f(n) \sim g(n)$, *meaning that f and g grow with the same rate*, if;
 - $\lim_{n \rightarrow \infty} f(n)/g(n) = \langle A \text{ constant value} \rangle$
- **Definition:**
 - If there is a positive constant so that the following equation holds:
 - $f(n) \sim c * g(n)$
 - Then the order of growth of the two functions ***f and g are the same.***
 - or, in other words:
 - The order of growth classification of ***$f(n)$ is $g(n)$.***
- “Order of growth” definition ignores leading coefficients and lower order terms in a function.

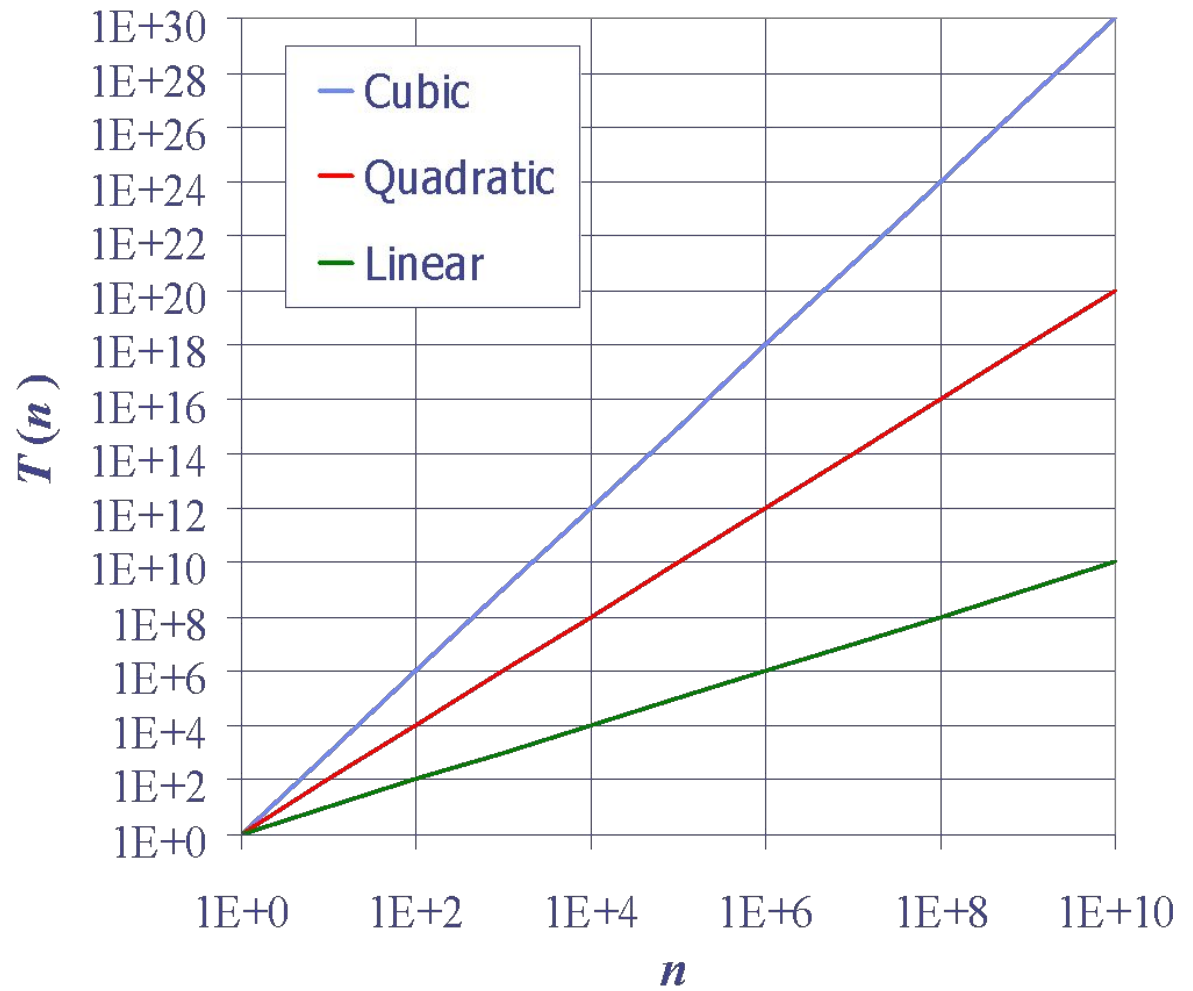
Values of some important functions as $n \rightarrow \infty$

n	$\log_2 n$	n	$n \log_2 n$	n^2	n^3	2^n	$n!$
10	3.3	10^1	$3.3 \cdot 10^1$	10^2	10^3	10^3	$3.6 \cdot 10^6$
10^2	6.6	10^2	$6.6 \cdot 10^2$	10^4	10^6	$1.3 \cdot 10^{30}$	$9.3 \cdot 10^{157}$
10^3	10	10^3	$1.0 \cdot 10^4$	10^6	10^9		
10^4	13	10^4	$1.3 \cdot 10^5$	10^8	10^{12}		
10^5	17	10^5	$1.7 \cdot 10^6$	10^{10}	10^{15}		
10^6	20	10^6	$2.0 \cdot 10^7$	10^{12}	10^{18}		

Values (some approximate) of several functions important for analysis of algorithms

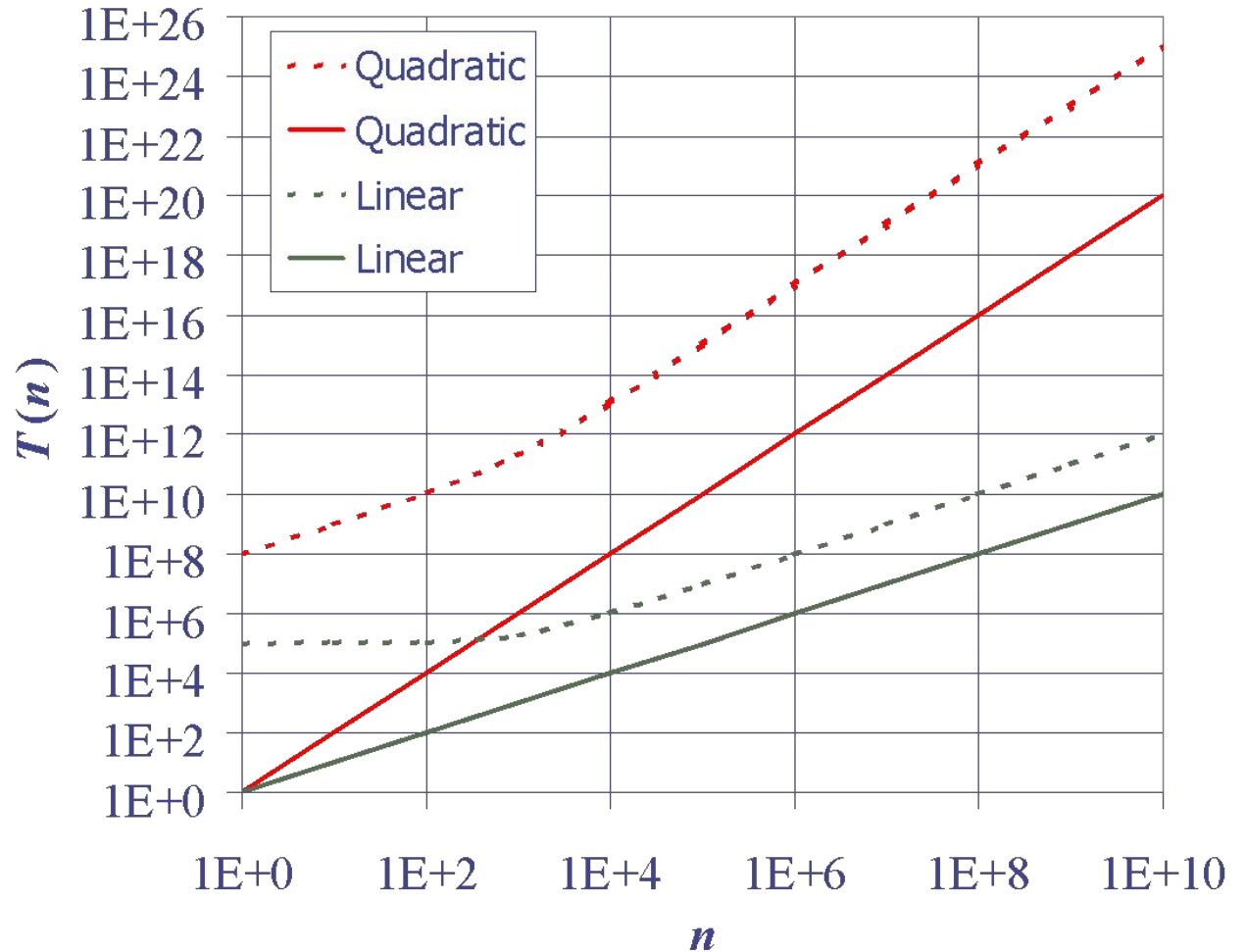
Growth Rates

- Growth rates of functions:
 - Linear $\approx n$
 - Quadratic $\approx n^2$
 - Cubic $\approx n^3$
- In a log-log chart, the slope of the line corresponds to the growth rate of the function



Constant Factors

- The growth rate is not affected by
 - constant factors or
 - lower-order terms
- Examples
 - $10n + 10^5$ is a linear function
 - $10^5 n^2 + 10^8 n$ is a quadratic function



Big-Oh Notation

- Given functions $f(n)$ and $g(n)$, we say that

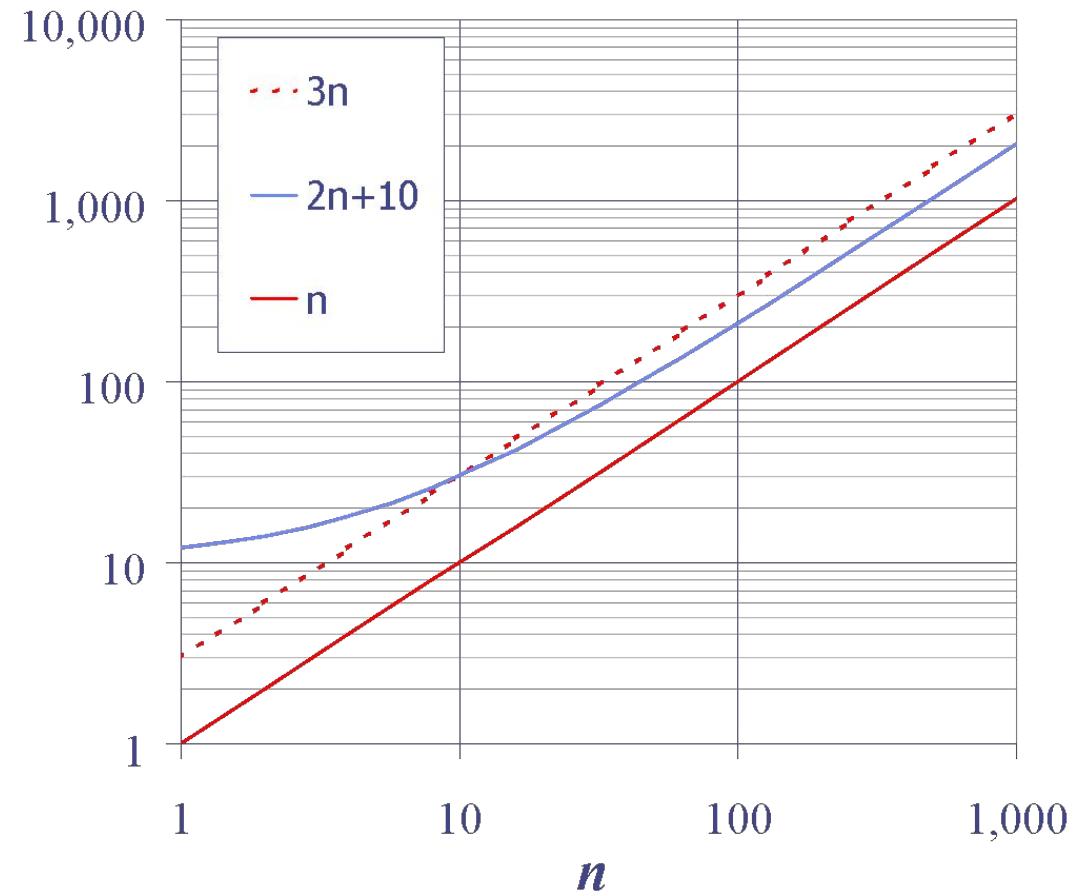
$f(n)$ is $O(g(n))$

if there are positive constants c and n_0 such that

$$f(n) \leq cg(n) \text{ for } n \geq n_0$$

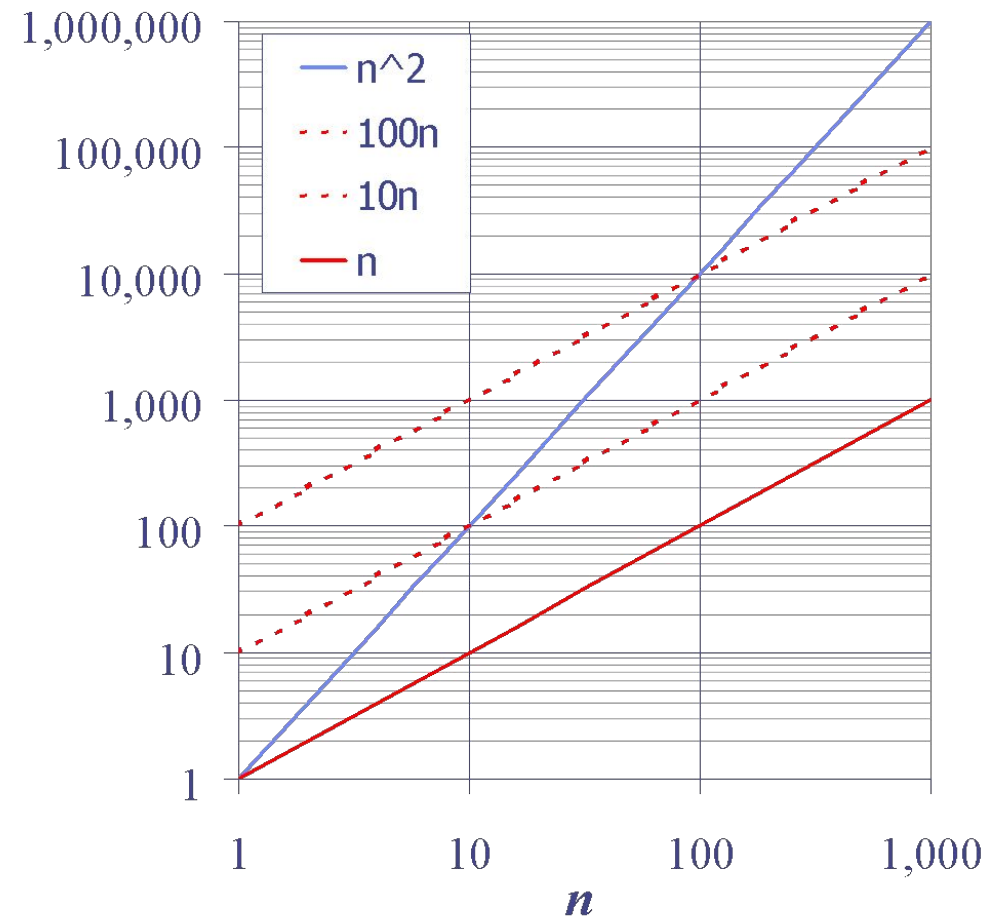
- Example: $2n + 10$ is $O(n)$

- $2n + 10 \leq cn$
- $(c - 2)n \geq 10$
- $n \geq 10/(c - 2)$
- Pick $c = 3$ and $n_0 = 10$



Big-Oh Example

- Example: the function n^2 is **not** $O(n)$
 - $n^2 \leq cn$
 - $n \leq c$
 - The above inequality cannot be satisfied since c must be a constant



More Big-Oh Examples

■ $7n-2$

$7n-2$ is $O(n)$

need $c > 0$ and $n_0 \geq 1$ such that $7n-2 \leq c \cdot n$ for $n \geq n_0$

this is true for $c = 7$ and $n_0 = 1$

■ $3n^3 + 20n^2 + 5$

$3n^3 + 20n^2 + 5$ is $O(n^3)$

need $c > 0$ and $n_0 \geq 1$ such that $3n^3 + 20n^2 + 5 \leq c \cdot n^3$ for $n \geq n_0$

this is true for $c = 4$ and $n_0 = 21$

■ $3 \log n + \log \log n$

$3 \log n + \log \log n$ is $O(\log n)$

need $c > 0$ and $n_0 \geq 1$ such that $3 \log n + \log \log n \leq c \cdot \log n$ for $n \geq n_0$

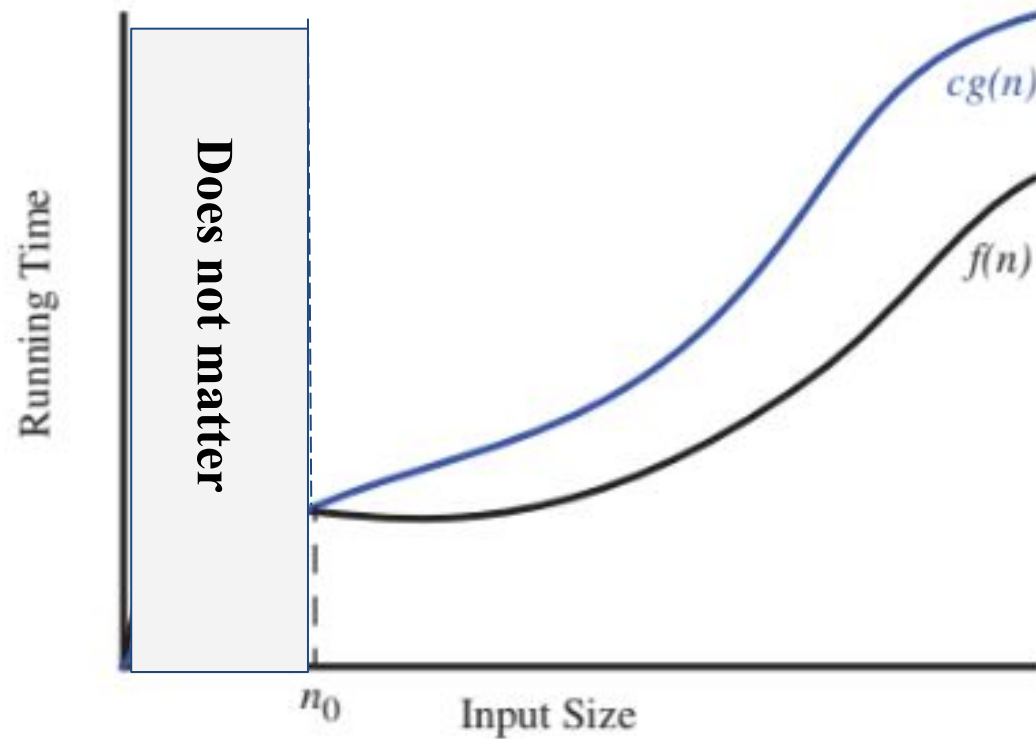
this is true for $c = 4$ and $n_0 = 2$

Big-Oh and Growth Rate

- The big-Oh notation gives an upper bound on the growth rate of a function
- The statement " $f(n)$ is $O(g(n))$ " means that the growth rate of $f(n)$ is no more than the growth rate of $g(n)$
- We can use the big-Oh notation to rank functions according to their growth rate

	$f(n)$ is $O(g(n))$	$g(n)$ is $O(f(n))$
$g(n)$ grows more	✓	✗
$f(n)$ grows more	✗	✓
Same growth	✓	✓

Big-oh (O)



The “big-Oh” notation. The function $f(n)$ is $O(g(n))$, since $f(n) \leq c \cdot g(n)$ when $n \geq n_0$.

Big-Oh Rules

- If $f(n)$ is a polynomial of degree d , then $f(n)$ is $O(n^d)$, i.e.,
 1. Drop lower-order terms
 2. Drop constant factors
- Use the "smallest possible" class of functions
 - Say " $2n$ is $O(n)$ " instead of " $2n$ is $O(n^2)$ "
- Use the simplest expression of the class
 - Say " $3n + 5$ is $O(n)$ " instead of " $3n + 5$ is $O(3n)$ "

Asymptotic Algorithm Run-Time Analysis

- The asymptotic analysis of an algorithm determines the running time in *big-Oh* notation.
- Also called *time complexity* of the algorithm.
- To perform the asymptotic analysis:
 - We find the number of primitive operations executed as a function of the input size (worst, best, average)
 - We express this function with *big-Oh* notation
- Example:
 - We determine that algorithm *arrayMax* executes at most $7n - 1$ primitive operations
 - We say that algorithm *arrayMax* "runs in $O(n)$ time"
- Since constant factors and lower-order terms are eventually dropped anyhow, we can disregard them when counting primitive operations.

Basic asymptotic efficiency classes

1	constant
$\log n$	logarithmic
n	linear
$n \log n$	n-log-n or linearithmic
n^2	quadratic
n^3	cubic
2^n	exponential
$n!$	factorial
n^n	?????

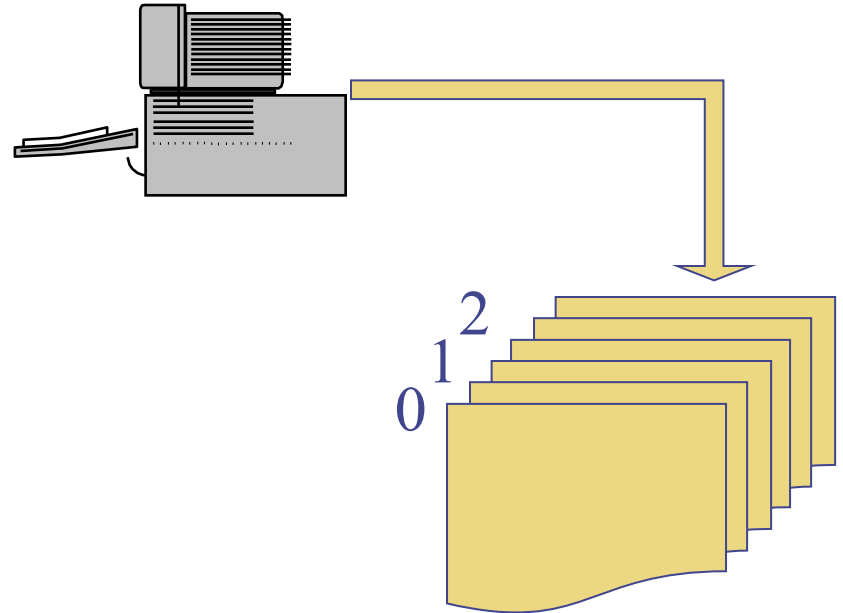
Estimating Memory Usage

- Any algorithm implemented in any programming language needs some memory to hold the algorithm's code.
 - But, the memory region occupied by the program's (or algorithm's) code does not depend on the size of the input(n).
- What depends on the size of the input with respect to memory usage is the data used by the algorithm.
 - Data usage by an algorithm is defined by the variables defined and used in the algorithm.
 - Input to an algorithm is also data, and it should be stored somewhere in memory.
 - Hence, input to an algorithm also contributes to the memory usage of the algorithm.

Estimating Memory Usage

- Our RAM model of computing tells us that each scalar variable needs one logical cell/slot of memory to store its value.
- Consequently, collection of values (sequences, arrays, sets, etc.) need memory space proportional to the size (n) of the collection.

- CPU



Estimating Memory Usage

Algorithm *arrayMax* ($A[0..n-1]$)

currentMax $\leftarrow A[0]$

for $i \leftarrow 1$ **to** $n - 1$ **do**

if $A[i] > \textit{currentMax}$ **then**

currentMax $\leftarrow A[i]$

return *currentMax*

memory units/slots

n (1 cell/slot for each array element)

1 (1 cell/slot for *currentMax*)

1 (1 cell/slot, for loop variable i)

Total $n + 2$

Note that since no additional variable is introduced at some lines, no extra logical memory cell/slot is occupied corresponding to those lines.

Estimating Running Time

- Algorithm *arrayMax* needs $n + 2$ memory cells/slots.
Define:
 - a = Actual memory needed by the variable that needs minimum memory units.
 - b = Actual memory needed by the variable that needs maximum memory units.
- Let $S(n)$ be worst-case memory space requirement of *arrayMax*. Then
$$a(n+2) \leq S(n) \leq b(n+2)$$
- Hence, the space requirement/complexity $S(n)$ is bounded by two linear functions.

Asymptotic Algorithm Space Complexity Analysis

- The asymptotic analysis of an algorithm's memory space complexity determines the memory space needed by the algorithm in big-Oh notation
- To perform the asymptotic analysis
 - We find the total number of logical memory cells/slots required by the algorithm to operate.
 - We express this function with big-Oh notation
- Example:
 - We determined that algorithm *arrayMax* requires at $n + 2$ logical memory slots
 - We say that algorithm *arrayMax*'s "space complexity is $O(n)$ "
- Since constant factors and lower-order terms are eventually dropped anyhow, we can disregard them when calculating space complexity.

Example 1 : Element uniqueness problem

ALGORITHM *UniqueElements*($A[0..n - 1]$)

//Determines whether all the elements in a given array are distinct

//Input: An array $A[0..n - 1]$

//Output: Returns “true” if all the elements in A are distinct

// and “false” otherwise

for $i \leftarrow 0$ **to** $n - 2$ **do**

for $j \leftarrow i + 1$ **to** $n - 1$ **do**

if $A[i] = A[j]$ **return false**

return true

Problem size: n

Basic operation: assignment or comparison

Time complexity: $T(n)$ is $O(n^2)$ (Worst Case-to return false in case no duplicates in the array),
is $O(1)$ (Best Case, when the first two elements of the array are equal)
? (Average Case, needs probabilistic analysis based on a set of assumptions)

Space Complexity : $S(n)$ is $O(n)$.

Example 2 : Matrix multiplication

ALGORITHM *MatrixMultiplication*($A[0..n-1, 0..n-1]$, $B[0..n-1, 0..n-1]$)

//Multiplies two n -by- n matrices by the definition-based algorithm
//Input: Two n -by- n matrices A and B
//Output: Matrix $C = AB$

```
for  $i \leftarrow 0$  to  $n - 1$  do
    for  $j \leftarrow 0$  to  $n - 1$  do
         $C[i, j] \leftarrow 0.0$ 
        for  $k \leftarrow 0$  to  $n - 1$  do
             $C[i, j] \leftarrow C[i, j] + A[i, k] * B[k, j]$ 
return  $C$ 
```

Problem size: n

Basic operation: assignment (or addition, or multiplication, or both)

Time complexity: $T(n)$ is $O(n^3)$ (Worst, Best, Average)

Space Complexity: $S(n)$ is $O(n^2)$

Example 3: Polynomial Evaluation

Problem: Find the value of polynomial

$$p(x) = a_n x^n + a_{n-1} x^{n-1} + \dots + a_1 x^1 + a_0$$

at a point $x = x_0$

Brute-force algorithm

Algorithm *Polynomial*($a[0..n], x$)

//Calculates the value of a given polynomial for given x

//**Input:** The coefficients of the polynomial and variable x

//**Output:** The resulting value of the polynomial

$p \leftarrow 0.0$

for $i \leftarrow n$ **downto** 0 **do**

$power \leftarrow 1$

for $j \leftarrow 1$ **to** i **do** //compute x^i

$power \leftarrow power * x$

$p \leftarrow p + a[i] * power$

return p

Time Complexity: $T(n)$ is

Example 4 : Polynomial Evaluation: Improvement

We can do better by evaluating from right to left:

Better brute-force algorithm

Algorithm *Polynomial*($a[0..n], x$)

//Calculates the value of a given polynomial for given x

//**Input**: The coefficients of the polynomial and variable x

//**Output**: The resulting value of the polynomial

$p \leftarrow a[0]$

$power \leftarrow 1$

for $i \leftarrow 1$ **to** n **do**

$power \leftarrow power * x$

$p \leftarrow p + a[i] * power$

return p

Time Complexity:

Example 5: Knapsack Problem

Given n items:

- weights: $w_1 \ w_2 \ \dots \ w_n$
- values: $v_1 \ v_2 \ \dots \ v_n$
- a knapsack of capacity W

Find most valuable subset of the items that fit into the knapsack

Example: Knapsack capacity $W=16$

item	weight	value
1	2	\$20
2	5	\$30
3	10	\$50
4	5	\$10



Knapsack Problem by Exhaustive Search

<u>Subset</u>	<u>Total weight</u>	<u>Total value</u>
{1}	2	\$20
{2}	5	\$30
{3}	10	\$50
{4}	5	\$10
{1,2}	7	\$50
{1,3}	12	\$70
{1,4}	7	\$30
{2,3}	15	\$80
{2,4}	10	\$40
{3,4}	15	\$60
{1,2,3}	17	not feasible
{1,2,4}	12	\$60
{1,3,4}	17	not feasible
{2,3,4}	20	not feasible
{1,2,3,4}	22	not feasible

Complexity :
T(n) is $O(2^n)$